

Serializability & Concurrency Control -- DBMS

Definition: Serializability is the correctness criterion for concurrent transaction execution. A non-serial schedule is serializable if its execution outcome is equivalent to executing the transactions sequentially one after another (a Serial Schedule).

1. Detailed Technical Explanation

1. Types of Schedules:

* Serial Schedule: Transactions execute strictly one after another (No interleaving). Guaranteed to be correct, but offers poor CPU utilization.

2. Conflict Serializability:

Two operations O_i and O_j in a schedule are in conflict if and only if:

Conflicting Operations Matrix:

<code>`Read(X)`</code>	<code>`Read(X)`</code>	<code>**No Conflict**</code>
<code>`Read(X)`</code>	<code>`Write(X)`</code>	<code>**CONFLICT**</code>
<code>`Write(X)`</code>	<code>`Read(X)`</code>	<code>**CONFLICT**</code>
<code>`Write(X)`</code>	<code>`Write(X)`</code>	<code>**CONFLICT**</code>

3. Precedence Graph (Serialization Graph) Test:

A directed graph $G = (V, E)$ used to test Conflict Serializability:

PRECEDENCE GRAPH RULE:

- If Precedence Graph contains NO CYCLES -> Schedule is CONFLICT SERIALIZABLE.
- If Precedence Graph contains CYCLES -> Schedule is NOT Conflict Serializable (Concurrency Anomaly!).

Example (Cycle detected):

```
[ T1 ] ---- Read(X), Write(X) ----> [ T2 ]
    ^                               |
    |----- Write(Y), Read(Y) -----| (Cycle! Not Serializable)
```

2. Core Concepts & Memory Keywords

- * Conflict Equivalent: Two schedules can be transformed into one another by swapping non-conflicting operations.

3. Must-Write Points for Exams

- * A schedule is conflict serializable if it is conflict equivalent to some serial schedule.

4. Quick Recall Flow

Concurrent Operations -> Identify Conflicts (R-W, W-R, W-W) -> Draw Precedence Graph -> No Cycles -> Serializable!